USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

Save what must be saved: Secure context switching with Sailor

Neelu S. Kalani, *EPFL and IBM Research - Zurich;* Thomas Bourgeat, *EPFL;*
Guerney D. H. Hunt, *IBM T. J. Watson Research Center;*
Wojciech Ozga, *IBM Research - Zurich*

https://www.usenix.org/conference/usenixsecurity25/presentation/kalani

This paper is included in the Proceedings of the
34th USENIX Security Symposium.

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the
34th USENIX Security Symposium is sponsored by USENIX.

ARTIFACT EVALUATED
usenix ASSOCIATION
AVAILABLE

Save what must be saved: Secure context switching with Sailor

Neelu S. Kalani *
EPFL & IBM Research — Zurich

Thomas Bourgeat
EPFL

Guerney D. H. Hunt
IBM T. J. Watson Research Center

Wojciech Ozga
IBM Research — Zurich

Abstract

Instruction set architectures (ISAs) are complex, with hundreds of registers and instructions that can modify dozens of them during execution, variably on each instance. Prose-style ISA specifications struggle to capture these intricacies of the ISAs, where often the important details about a single register are spread out across hundreds of pages of documentation. Ensuring that all ISA-state is swapped in context switch implementations of privileged software requires meticulous examination of these pages. This manual process is tedious and error-prone.

We propose a tool called Sailor that leverages machine-readable ISA specifications written in Sail and Isla symbolic execution engine for Sail to automate this task. Sailor determines the ISA-state that must be swapped during the context switch using the data collected from Isla and a novel algorithm to identify all the ISA-state that must be swapped during context switches. We use Sailor to assess the context switch code of multiple systems: from regular user process context switching code in the RISC-V Linux kernel on the StarFive VisionFive2 board, to the enclave context switching code in confidential computing frameworks, Keystone and Komodo. We identify multiple mishandled security-sensitive ISA-state. This research exposes an often overlooked attack surface that stems from mishandled ISA-state, enabling unprivileged adversaries to exploit system vulnerabilities.

1 Introduction

The increasing complexity of architectures, and the nitty-gritty details of the Instruction Set Architectures (ISA) being scattered throughout hundreds or thousands of pages have put system security at risk. A simple overlook of a single register in security-critical code can inadvertently expose the system to security vulnerabilities. Low-level software developers need to do the tedious job of manually navigating through the dense ISA specifications to ensure they do not introduce any security vulnerabilities in their code. Moreover, developers need to stay up-to-date with the evolving ISA specifications and the hardware platforms that support them.

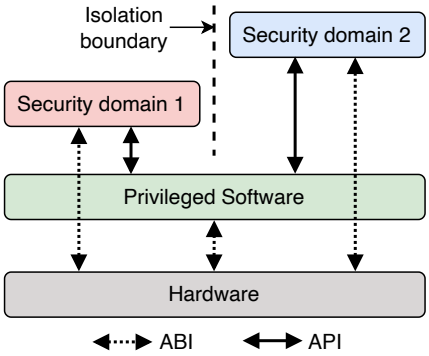

Figure 1: Security domains isolated by a privileged software that mediates the context switch between the domains. The security domains access the ISA-state through the Application Binary Interface (ABI). The privileged software must swap all security-sensitive ISA-state to prevent attacks.

ISAs have traditionally been specified using hand-written semantics in natural language, occasionally accompanied by pseudo-code, in bulky and comprehensive documents. For example, the Arm A-profile architecture reference manual [22] and the privileged and unprivileged RISC-V ISA manuals [14] consist of 15,000 and 900 pages, respectively. These ISA manuals are highly technical and are becoming increasingly dense, with new features and instructions added in every version. For instance, the Arm v7-A, v8-A and v9-A consist of 2700, 5200, and 15,000 pages, respectively [4, 21, 22]. Similarly, on RISC-V, the non-ISA specifications (interrupt architecture, IOMMU, etc.) further add 767 pages [7], and new ISA extensions are ratified every year [13]. Prior work shows a similar trend for the x86 ISA [25].

New features introduce new ISA-state (general purpose registers (GPRs) or control and status registers (CSRs)), adding

*Majority of the work was done while the author was at IBM Research — Zurich.

more complexity to the already convoluted ISA-state, especially when it comes to isolating software. Figure 1 shows two different security domains that execute in a system. A privileged software mediates the context switch between the two security domains. To the security domains, it should appear as if they are the only software that executes in the system [30, 38]. There are two layers at which this guarantee needs to be enforced: the application programming interface (API) and the application binary interface (ABI). At the API-layer, which enables interaction between the different security domains, inputs from untrusted sources requires sanitization to prevent Iago-style attacks [28]. Through the ABI-layer, the security domains can access a significant part of the ISA-state. This involves 1000+ and 140+ registers on Arm v9-A [16] and RISC-V [17], respectively. The privileged software must appropriately reset the ISA-state before transferring control to the second security domain with two goals: no data of the first security domain must leak to the second one, and that the ISA-state changes by first security domain do not affect execution of the second one. We term the set of ISA-state that can leak data or impact execution integrity across security domains as security-sensitive.

In this paper, we focus on the ABI-layer, improper sanitization at which can introduce various security vulnerabilities. Prior work has discovered security vulnerabilities at the ABI-layer in industry-standard software as well as academic research projects. These involve improper flags sanitization [52] in Intel SGX-SDK [9] and Microsoft's open enclave [12] runtimes, register leakage [26] and ROP [40] on Intel SGX-LKL [47] (another SGX runtime), interrupt hijacking [6] on Intel TDX [10], side-channels and computational integrity attacks through the floating point unit on Intel SGX and RISC-V Keystone [18, 39]. Small oversights (of even a single bitfield in a register) in the context switch implementation introduce high-impact vulnerabilities that lead to simple and cost-effective attacks that even unprivileged adversaries can mount. The matter gets worse when dealing with ISAs such as RISC-V that promote adding new extensions and the privileged software inadequately tracks which ISA extensions are enabled by the platform vs. supported by the software [18].

Essentially, the attack surface introduced by improper sanitization of ISA-state (specifically, GPRs and CSRs) when switching across security domains is often overlooked. To approach this problem, we propose to perform symbolic execution of the machine-readable ISA specification called Sail [15] to automatically find all the security-sensitive ISA-state. With that, we aim to equip software developers with conveniently accessible insights to the ISA that eliminate the need to analyze hundreds or thousands of pages of manuals.

Sail is an industry-adopted solution to define ISA specifications. The RISC-V Foundation considers Sail as the official golden model and encourages that ratified extensions are defined in it [17]. Arm uses Arm specification language (ASL) and has tools to automatically translate it to Sail [16]. Sail models are complete enough to generate emulators that boot Linux and are used for formal verification [15]. Sail also enabled the creation of sophisticated works in the formal methods space such as Isla [23] and Islaris [48]. Isla is a symbolic execution engine for Sail code that uses the Z3 SMT Solver. Isla converts the Sail model into an intermediate representation that it uses for the symbolic execution [8]. Given as input an instruction along with SMT constraints on the ISA-state *e.g.,* the current privilege level for execution, Isla produces traces of instruction execution that describes the instruction's dependencies and effects on the ISA-state.

We introduce Sailor [1], a framework that automatically computes the set of security-sensitive ISA-state by leveraging Isla. The security-sensitive ISA-state can leak information or impact execution integrity of security domains, and thus, must be swapped during context switches. We apply Sailor on the Sail model for the RISC-V architecture [17] to comprehensively identify all the ISA-state that must be swapped during context switching. This includes extracting intricate details about the ISA such as side-effects of instruction execution on the ISA-state. For example, exceptions that occur during floating-point instruction execution on RISC-V update the `fflags` CSR as a side-effect. Prior work demonstrated a side-channel attack where an adversary infers the victim's floating-point operations and exceptions by observing the `fflags` CSR [18]. While prior work relied on manual ISA specification inspection to determine that `fflags` can leak data and must be swapped during context switches, Sailor automates the discovery of such information.

We use this information to perform security analysis of (i) context switching across regular user processes running on top of the Linux distribution dispatched with the StarFive VisionFive 2 [2], and (ii) context switching across different security domains (untrusted OS and enclave) in two open-source confidential computing systems i.e. Keystone [39] and Komodo [29]. We find multiple mishandled ISA-state in the context switch implementations of these systems. Prior work previously reported two bugs in the context switch implementation of the Keystone framework [18] by performing manual analysis of the ISA specification to determine which state should be swapped. Our tool automatically identifies these ISA-state as security-sensitive. We further include automatic test generation of user programs that read/write the security-sensitive ISA-state and check whether the state was swapped across context switching or not.

Our main contributions are:

- We provide a new framework called Sailor that leverages the Isla symbolic execution tool for Sail to automatically inspect the ISA specifications and collect ISA-level insights (Subsection 3.1).
- We further provide a novel algorithm to use the ISA-level

[1] Artifact available at: https://doi.org/10.5281/zenodo.15613681

insights to compute the set of ISA-state that must be swapped during context switches *i.e.,* security-sensitive ISA-state (Subsection 3.2).

- We evaluate the context switch implementation in three systems - Linux shipped by StarFive for the VisionFive2 board, and two open-source confidential computing frameworks Keystone and Komodo, using the results of the algorithm. We also provide automatic test generation for the evaluation on Linux (Subsection 3.3). We report multiple mishandled ISA-state in these systems (Subsection 5.3).

Responsible disclosure: We have disclosed the bugs and insights discovered through this work for Keystone and StarFive VisionFive2 Linux via github issues on the appropriate repositories [3, 11]. [2]

2 Motivation & Background

2.1 ISAs are Complex

ISA manuals are increasingly dense and ISA-state (architectural context), increasingly convoluted. It primarily comprises of general purpose registers (GPRs), control and status registers (CSRs), and memory. There is a plethora of research on keeping memory of different security domains confidential using various access control mechanisms [14, 19, 20, 32, 51]. Yet, little focus has been put towards preventing security vulnerabilities that arise from improper sanitization of ISA-state when switching across security domains [52]. However, high-impact vulnerabilities can be trivially introduced in context switch implementations while swapping ISA-state, that can further be exploited by an unprivileged adversary with minimal effort. The reason for this is two-fold.

First, control and status registers (CSRs in RISC-V), special-purpose registers (Arm) or model specific registers (MSRs in x86) are immensely complex - up to the bit-level. Each bit in such registers can reconfigure the processor's behavior, thus influencing the execution of the following instructions. It is trivial to introduce bugs when setting/clearing some of the bits in a register due to possible inter-dependencies with bits of another register. For instance, when any floating point (FP) operations are executed on RISC-V platforms, a Dirty bit is set in the `sstatus` register as an optimization to check whether the floating point register state needs be saved. However, not clearing this bit during a context switch can leak information about floating point unit use inside a security domain. Similarly, prior work reported that Intel SGX's SDK fails to clear two bits during the context switch that harm the computational integrity of the operations executed inside the enclave and also lead to memory corruption [52].

[2] For Komodo, the development is inactive since more than 3 years.

Second, there is a higher risk of introducing vulnerabilities at the ISA-level for ISAs that promote modularity and customization, such as RISC-V. This is especially true if the privileged software does not properly handle discovery of the ISA extensions supported by the platform on which it runs. Prior work shows that incorrect sanitization of floating point state on the x86-64 and RISC-V architectures leads to unintentional side-channels that can be exploited by unprivileged adversaries [18]. Moreover, some of the mishandled ISA-state we find also stem from ISA-extensions.

2.2 Overlooking security at the ABI-layer

Existing strategies to avoid vulnerabilities focus on preventing software bugs at the application level (e.g. application binary scanning, symbolic execution of user code, etc.) or introduce patches to prevent sophisticated micro-architectural attacks (speculative execution, cache side-channels) [35, 42, 55]. These techniques do not target bugs at the ABI-layer that can breach the security guarantees of security domains even when the application code is secure and caches are partitioned.

Further, formal verification of privileged software (that targets to avoid attacks at the ABI-layer too) relies on self-written specifications (rather than official ISA specifications) that usually model a subset of the entire machine [29, 30, 45]. This is important to make the verification efforts feasible, by limiting and pruning the paths that need to be explored. These partial machine models are used to prove functional correctness of the security monitors *i.e.,* whether the context switch is saving and restoring the registers (to and from memory) as expected. They are also used to prove the non-interference security property *i.e.,* on switches from security domain A to security domain B and then back to security domain A, the security domain A must not be able to distinguish the system state before vs. after the execution of security domain B. However, the verification only proves properties that hold for the ISA-state modeled by the self-written specifications, instead of the entire ISA-state. In such cases, it's difficult to extend the proven properties to the ISA-state that was not modeled. It is also difficult to identify security-sensitive ISA-state in the first place, which could be useful to determine the exact subset of the ISA that the verification frameworks must model. However, that requires studying the ISA specifications extensively and meticulously.

2.3 Secure Context Switches

A naive approach to implement secure context switches is to consider all ISA-state to be intrinsically security-sensitive. Thus, a context switch implementation in privileged software would swap the entire ISA-state when switching across security domains. While in theory this approach seems intuitive and appealing, there are two problems with this approach.

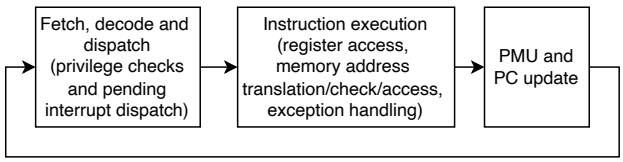

Figure 2: Flow block diagram of the RISC-V Sail Model implementation

First, due to the convoluted nature of architectures, with hundreds or thousands of registers [14,22], it is trivial to overlook ISA-state that requires to be swapped during context switches across security domains. This is especially true in the case of ISA-extensions. It is because of such oversights that security vulnerabilities get a chance to surface-up and be exploited by unprivileged adversaries with minimal efforts [18,52].

Second, context switches are expensive [31,41]. Context switching across security domains doesn't just involve reading and writing of registers, but also saving the current context onto memory and reading the previously stored context from memory. Further, depending on the ISA-state that is exposed to a security domain (executing in a particular privilege mode) and the ISA-state that the execution of the security domain is dependent on, not all ISA-state might be security-sensitive. For example, there are three privilege modes on RISC-V *i.e.,* User (U-mode), Supervisor (S-mode) and Machine (M-mode). If two security domains execute in U-mode, then the S-mode CSR `satp` would be considered security-sensitive as it contains the page table root address, that would impact the execution of the security domains in U-mode. However, the S-mode CSR `sepc` that contains the exception program counter on traps from U-mode to S-mode, would not be considered security-sensitive since it neither impacts execution of the security domains in U-mode, nor is accessible in U-mode. It can be beneficial especially for performance-critical systems to dig deeper in the ISA and find out exactly which ISA-state needs to be swapped during a context switch.

2.4 Sail & Isla

The Sail model specifies the encoding, decoding and the instruction semantics for each instruction in the ISA. Figure 2 shows the block diagram for the instruction fetch till execute loop in the RISC-V Sail implementation [17]. Each instruction is first fetched, decoded and then dispatched. Before the instruction semantics are executed, two things are checked: (i) if any interrupts are pending, which might be serviced before executing the instruction, and (ii) if the privilege level in the current architectural context allows executing the instruction.

Each instruction (or sometimes group of the same type of instructions, depending on the Sail model implementation) has an independent definition specified in the Sail model implementation. This definition of the instruction is then executed, which may involve accessing memory (and corresponding memory translations and checks), performing arithmetic operations, or modifying the processor state (registers). The execution may also lead to exceptions, in which case the appropriate handler must be called. Once the execution is complete, the performance monitoring unit (PMU) and the PC are updated, and the procedure is repeated for the next instruction.

We will now deep dive into a small Sail snippet to understand how instruction specifications are implemented in Sail. The following text is taken from the RISC-V privileged ISA specification manual [14], which describes the `mret` instruction.

"An MRET or SRET instruction is used to return from a trap in M-mode or S-mode respectively. When executing an xRET instruction, supposing xPP holds the value y, xIE is set to xPIE; the privilege mode is changed to y; xPIE is set to 1; and xPP is set to the least-privileged supported mode (U if U-mode is implemented, else M). If y≠ M, xRET also sets MPRV=0."

```
function clause execute MRET() = {
  if   cur_privilege != Machine
  then { handle_illegal(); RETIRE_FAIL }
  else {
    mstatus[MIE]  = mstatus[MPIE];
    mstatus[MPIE] = 0b1;
    cur_privilege = privLevel(mstatus[MPP]);
    let prev_priv = if extEnabled(Ext_U)
                    then User else Machine);
    mstatus[MPP]  = bits(prev_priv);
    if   cur_privilege != Machine
    then mstatus[MPRV] = 0b0;
    PC = prepare_xret_target(Machine)
    RETIRE_SUCCESS
  }
}
```

Listing 1: Sail implementation of `mret` in RISC-V.

Listing 1 shows the corresponding Sail implementation of the `mret` instruction. The instruction definition is implemented using the "function clause execute <name>()". The instruction execution is allowed only if the privilege mode in the current architecture context is set to machine mode (M-mode). The exception handler is then called to execute the "CTL_MRET" case (depicted on the right side of the figure). Here, we can see the translation of the prose-style specification listed above into machine-readable specification in Sail. For instance, `MIE` (machine interrupt enable) is set to `MPIE` (machine previous interrupt enable) and the privilege level is set to the value in `MPP` (Machine previous privilege level) in the `mstatus` register.

The Sail implementation thus describes, in machine-

readable format, every single effect on the ISA-state of executing an instruction, given the access rights of a particular privilege mode.

Isla, is a Sail addon that can perform symbolic execution on a Sail model. Isla produces instruction execution traces that describe the effects of instruction execution on the ISA-state in an SMT-LIB2-like format [24]. Isla prunes exploration paths by eliminating unreachable paths of the ISA specification using the Z3 SMT solver. Given an instruction and ISA-state constraints (e.g., privilege mode), Isla generates traces capturing the effects of instruction execution, including the CSRs that the instruction reads or writes.

```
1 (trace
2   (define-enum |Privilege| 3 (|User| |Supervisor| |Machine|))
3   (assume-reg |cur_privilege| Machine |)
4   (read-reg |PC| v442)
5   (read-reg |cur_privilege| Machine |)
6   (read-reg |mstatus| (struct (|bits| #x0000000a00000000)))
7   (write-reg |mstatus| (struct (|bits| #x0000000a00000000)))
8   (read-reg |mstatus| (struct (|bits| #x0000000a00000000)))
9   (write-reg |mstatus| (struct (|bits| #x0000000a00000080)))
10  (read-reg |mstatus| (struct (|bits| #x0000000a00000080)))
11  (write-reg |cur_privilege| User |)
12  (read-reg |mstatus| (struct (|bits| #x0000000a00000080)))
13  (read-reg |misa| (struct (|bits| #x8000000000141101)))
14  (write-reg |mstatus| (struct (|bits| #x0000000a00000080)))
15  (read-reg |cur_privilege| User |)
16  (read-reg |mstatus| (struct (|bits| #x0000000a00000080)))
17  (write-reg |mstatus| (struct (|bits| #x0000000a00000080)))
18  (read-reg |mepc| #x0000000000000000)
19  (read-reg |misa| (struct (|bits| #x8000000000141101)))
20  (write-reg |nextPC| #x0000000000000000)
21  (define-enum |Retired| 2 (|SUCCESS| |FAIL|))
22 )
```

Listing 2: The SMT-LIB2 trace generated by Isla for the MRET instruction executed in M-mode.

Listing 2 shows the trace generated by Isla for the execution of the `mret` instruction in M-mode. Along with the SMT-LIB2 semantics, there are additional annotations in the trace to describe the events meaningfully *e.g.,* `read-reg` or `write-reg` for a register read or write. The traces from Isla also capture if a trap occurs during the execution. For example, Listing 3 shows the execution of the `mret` instruction in S-mode, which is illegal according to the ISA specification (Listing 1). As shown in the trace, after reading that the `cur_privilege` is S-mode (line 5 in the trace), the exception delegating register on RISC-V (`medeleg`) is read, the `mcause` register is written with the value 2 (line 9) which denotes an illegal instruction trap. More trap information is saved in the `mepc` and the `mtval` registers, and `cur_privilege` is updated to M-mode so the trap can be handled in M-mode.

```
1 (trace
2   (define-enum |Privilege| 3 (|User| |Supervisor| |Machine|))
3   (assume-reg |cur_privilege| Supervisor |)
4   (read-reg |PC| v442)
5   (read-reg |cur_privilege| Supervisor |)
6   (read-reg |medeleg| (struct (|bits| #x0000000000000000)))
7   (read-reg |misa| (struct (|bits| #x8000000000141101)))
8   (read-reg |mcause| (struct (|bits| #x0000000000000000)))
9   (write-reg |mcause| (struct (|bits| #x0000000000000002)))
10  (read-reg |mstatus| (struct (|bits| #x0000000a00000000)))
11  (read-reg |cur_privilege| Supervisor |)
12  (read-reg |mstatus| (struct (|bits| #x0000000a00000000)))
13  (write-reg |mstatus| (struct (|bits| #x0000000a00000800)))
14  (write-reg |mtval| #x0000000000000000)
15  (write-reg |mepc| v442)
16  (write-reg |cur_privilege| Machine |)
17  (read-reg |mcause| (struct (|bits| #x0000000000000002)))
18  (read-reg |mtvec| (struct (|bits| #x0000000000000000)))
19  (define-enum |TrapVectorMode| 3 (|TV_Direct| |TV_Vector|
20  |TV_Reserved|))
21  (write-reg |nextPC| #x0000000000000000)
22  (define-enum |Retired| 2 (|SUCCESS| |FAIL|))
23 )
```

Listing 3: The SMT-LIB2 trace generated by Isla for the MRET instruction executed in S-mode.

3 Sailor Design

Figure 3 shows the high-level overview of Sailor. It consists of two components: ISA Inspector and Analyzer. The goal of the framework is to enable secure context switching of ISA-state by privileged software that intends to enforce isolation between different security domains running on the system.

The ISA Inspector generates instruction execution traces from Isla and parses them to build *tables* that list all the ISA-state read and written by each instruction during execution, which privilege modes can access which ISA-state, and which privilege modes can execute which instructions. (examples given in Table 1, Table 3, Table 2).

The Analyzer then takes these tables and feeds them to the Classifier Algorithm (Subsection 3.2) that processes the insights to classify ISA-state as security-sensitive, that must be swapped during context switches.

We first use this information to automatically generate tests. The tests are simple user programs that update a flagged-to-be security-sensitive ISA-state and read it after a context switch to check if the value changed.

In RISC-V, there are special CSRs that cannot be directly read or written in lower privilege modes. For such CSRs that are also flagged as security-sensitive, we simply produce a list back to the developer. The developer can either write custom tests, or use utilities like grep to find out whether existing software of interest swaps the security-sensitive ISA-state in the codebase they want to assess the security of. In this scenario, Sailor can still serve as a compass to bring the developer's attention to where the code manages the security-sensitive ISA-state, if it exists, and check that it does so properly.

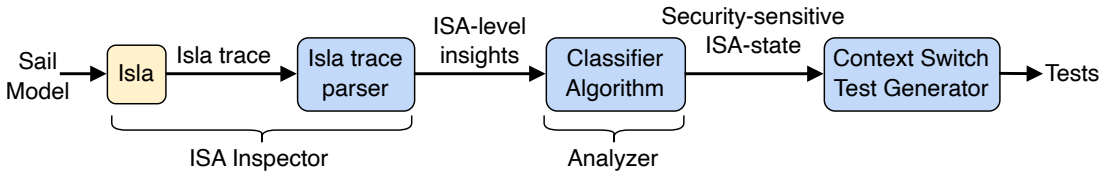

Figure 3: Sailor Overview

System engineers could also use the list of sensitive ISA-state generated by Sailor as a guide for writing privileged software. The algorithm's output could also be leveraged by ISA architects that design new ISA extensions as a guideline to ensure that the proposed extension does not affect the previously determined security-sensitivity of the ISA-state. We discuss this further in subsubsection 5.3.4.

3.1 ISA Inspector

The ISA Inspector automates the task of extracting ISA-level insights by leveraging the Isla symbolic execution engine for Sail. We start by informally outlining three questions Sailor first tackles. Each of them corresponds to the respective table (Table 1, Table 3, and Table 2) produced by the ISA Inspector.

1. Does a privilege mode have explicit access rights to read or write a particular ISA-state? The simplest method to establish a side-channel or a covert-channel is through shared ISA-state. For example, on RISC-V, confidential VMs that execute in VS-mode have the permission to explicitly write the `vsscratch` CSR; and hypervisors that execute in HS-mode have the permission to explicitly read the `vsscratch` CSR. Thus, if a security monitor does not clear the `vsscratch` CSR during a context switch, it can be leveraged to leak the confidential VM's security-sensitive information. In other words, this question checks whether a privilege mode can execute explicit ISA-state read/write instructions (*e.g.,* `csrrs` and `csrrw` in RISC-V from the Zicsr extension). The Analyzer uses the insights from this question (example output shown in Table 1) to determine whether a direct communication channel can be established between two different security domains.

2. What is the ISA-state footprint of an instruction? We define the ISA-state that could be read or written as a side-effect of the execution of an instruction as the *ISA-state footprint* of an instruction. For example, as shown in Listing 1, executing the `mret` instruction has several side-effects on the ISA-state, *e.g.,* updating the `MIE` field with the `MPIE` field's value in the `mstatus` CSR. This implies that, the `mret` instruction implicitly depends on the value of `MPIE` and updates the value of `MIE`. Table 2 shows an example output generated by the ISA inspector for answering this question. One important case to address while computing the ISA-state footprint of instructions is the false instruction dependence due to a `read-after-write`. This means that if an instruction first writes a particular ISA-state and then reads it back, it will never observe any value other than the one written during the current instruction's execution. Thus, we treat `read-after-write` scenarios simply as writes to prevent false positives.

The instruction footprint information is crucial as instructions implicitly (or indirectly) update ISA-state. When the context switch implementation does not clear such ISA-state, a security domain can infer information about the execution in another security domain. For example an adversary can infer whether the victim uses the floating point unit and whether any (and which) exceptions occur during execution [18]. We also include the interrupt handling, PMU and PC updates in the footprint (as shown in Figure 2). Prior work has shown that security vulnerabilities like interrupt hijacking can stem from overlooking even a few bits of a register in the context switch implementation [6]. Thus, it is crucial to appropriately handle security-sensitive ISA-state that changes as a side-effect of instruction execution during context switches.

3. Does a privilege mode have the access rights to execute a particular instruction? If a privilege mode does not have the access rights to execute an instruction (*e.g.,* only M-mode can execute the `mret` instruction, output example in Table 3) and ultimately never directly or indirectly reads or writes certain ISA-state, then that helps reduce the amount of ISA-state that needs to be swapped to perform a secure context switch. This information can be useful for performance-sensitive workloads, as context switches are known to be expensive [31, 41].

More systematically, Sailor operates by defining ISA-state, footprint and the notion of dependent instruction and dependent privilege mode as follows:

ISA-state: A Sail model defines a set of registers used by the ISA. These registers are both, GPRs and CSRs. Each register is typically given a name: `a0`, `mstatus`, `scounteren`, etc. Any of these Sail-defined registers, which can appear in an Isla trace with the "read-reg" or "write-reg" events, is named an ISA-state.

Footprint of a given instruction: An ISA-state is in the read (resp. write) footprint of a given instruction if there exists

ISA-State	User Read	User Write	Supervisor Read	Supervisor Write	Machine Read	Machine Write
mideleg	Not allowed	Not allowed	Not allowed	Not allowed	Allowed	Allowed
...	...	...	...	...	...	...

Table 1: Direct read and write access on ISA-state per privilege mode.

Instruction	User CSR R	User CSR W	Supervisor CSR R	Supervisor CSR W	Machine CSR R	Machine CSR W
JAL	misa, medeleg, mcause, mcause, mtvec, mstatus	mcause, mstatus, mtval, mepc	misa, medeleg, mcause, mcause, mtvec, mstatus	mcause, mstatus, mtval, mepc	misa, medeleg, mcause, mcause, mtvec, mstatus	mcause, mstatus, mtval, mepc
...	...	...	...	...	...	...

Table 2: CSR footprint per instruction per mode, simple example of Jump And Link instruction, with similar footprint across privilege modes.

an Isla trace for that instruction that reads (resp. writes) the said ISA-state. In other words, there exists an execution of that instruction that reads (resp. writes), the said register.

Dependent instruction and dependent privilege mode: We say that an instruction is dependent on an ISA-state if the said ISA-state is in the read footprint of that instruction. We say that a privilege mode is dependent on a given ISA-state if there exists an instruction executable in that privilege that is dependent on the said ISA-state.

3.2 Analyzer

Figure 4 shows the *Classifier algorithm* we introduce to determine which ISA-state is security-sensitive. This analysis must be repeated for each pair of privilege modes in order to determine the exact ISA-state required to be swapped on a security domain switch. In Figure 4, the term *Source* refers to the privilege mode in which the previous security domain had been executing, and *Target* refers to the privilege mode in which the next security domain will be executing.

The algorithm's goal is to identify any channels in the ISA-state that (i) allow the Source to affect the results of execution in the Target (adversarial Source), (ii) leak information of the Source to the Target (adversarial Target), or (iii) enable establishing communication (adversarial Source and Target). The analysis excludes GPRs since they can be written and read by any privilege mode and an adversary can easily leak data of another security domain through the GPRs (forming side-channels, covert-channels or computational integrity attacks), so they are considered security-sensitive by default. The analysis is repeated for each ISA-state (*e.g.,* a CSR).

To identify (i), the algorithm checks whether the ISA-state is explicitly or implicitly writable by the Source mode. Here, explicit access refers to the first question in Subsection 3.1 (*e.g.,* `csrw mstatus, in_reg` instruction in RISC-V). Whereas, implicit access refers to the side-effects captured in the ISA-state footprint of an instruction (*e.g.,* the `fmadd` instruction in RISC-V execution can update the `mstatus.FS` bits to Dirty). If a privilege mode has the rights to execute any instruction that can write the ISA-state as a side-effect of execution, it is considered to have implicit write access to the ISA-state. The algorithm further checks whether the execution in Target mode depends on the ISA-state (via explicit or implicit reads). This ISA-state is considered as security-sensitive because if it is not appropriately swapped during the context switch, it can constitute **computational integrity** attacks on the Target security domain, breaching its integrity guarantees. For example, the Source can maliciously modify the rounding mode of computations, or disable extensions used by the Target.

Similarly, to identify (ii) and (iii), the algorithm checks whether the Source mode can implicitly or explicitly write the ISA-state and whether the Target mode can implicitly or explicitly read that ISA state. If the criteria is satisfied, then the ISA-state is considered as security sensitive because either the Target domain can breach the confidentiality of the Source domain by reading security-sensitive data of the Source domain or inferring details about execution in the Source domain. That would constitute **side-channel** attacks on the Source domain. Further, this ISA-state can also be used to breach the isolation guarantees in the system, enabling two-way communication between the Source and Target (*i.e.,* **covert-channel**).

The algorithm performs one further check to identify (ii), *i.e.,* whether the ISA-state contains sensitive data of the Source that it depends on (implicitly reads) during execution (*e.g.,* `satp` CSR in RISC-V which holds the page table root address and is used implicitly during execution of load/store instructions). If the Target can read this ISA-state, it can again constitute side-channel attacks on the Source domain.

If no such channels are identified, the algorithm considers

Instruction	User Executable	Supervisor Executable	Machine Executable
MRET	Not allowed	Not allowed	Allowed
...	...	...	...

Table 3: Instruction Access per privilege mode.

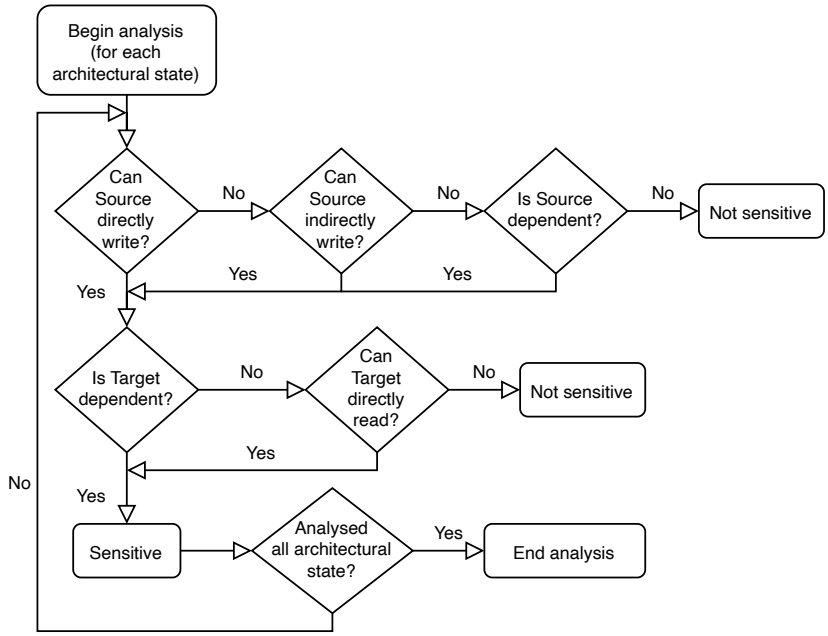

Figure 4: Algorithm to classify architectural state as security-sensitive. Source and Target correspond to privilege modes of the security domains involved in the context switch.

the particular ISA-state as not security-sensitive and moves on to analyze another ISA-state.

Despite the apparent simplicity of the questions that the algorithm asks to identify the potential information flow channels in the ISA-state, when combined with the fine-grained ISA-level insights that the Scanner generates, the Analyzer can help catch bugs that can constitute different classes of security vulnerabilities (such as computational integrity, side-channels, covert-channels).

Once we have information regarding which ISA-state is security sensitive and which isn't, we can validate the existing context switch implementations against that information.

3.3 Assessing the handling of sensitive ISA-state during context switch

We use the list of sensitive ISA-state generated by Sailor to assess if sensitive values are saved for these state across context switch.

In general, it is not trivial to fully and systematically automate the test for this property for all privilege modes and all sensitive ISA-state. This is because not all the CSRs are directly readable and writable from each privilege mode, and also because some CSRs can only hold specific values (Write-Any-Read-Legal (WARL) CSRs), *e.g.,* there are five legal values for the floating point rounding mode (frm) CSR i.e. 0 to 4, the remaining values are illegal. So writing the CSRs in the tests cannot simply consider writing random values, or else the write won't reflect and the test will not produce accurate results. So we follow a best-effort approach for the testing.

We especially focus on automatically generating tests for the user-to-user context switch. From the list of security sensitive ISA-state provided by the Analyzer, for each CSR flagged as security sensitive, we produce pairs of unit tests that attempt to (i) write a random value to a CSR of interest from one process (the first program) and (ii) read the CSR from another process to check against the random expected value (the second program). If the expected value is seen, we detect

that the CSR was indeed mishandled by the context switching code *i.e.,* was not cleaned up across context switches.

Providing a list of legal values for all WARL CSRs, our testing could be improved. For the CSRs that cannot be written and read directly, the checking of the code must be done manually or by instrumenting the context switch code, in case it is open source, or custom tests must be devised.

On top of our testing approach, we also emphasize the value of knowing what to look for: we found that having a list of sensitive register to grep for in the source code of an operating system is already practically helpful. Using a mix of these approaches we found multiple mishandled ISA-state across different context switching code (Section 5.3).

Further, in addition to finding bugs in the context switch code, the testing approach also provides confidence in the context switch code to swap the security sensitive ISA-state correctly.

4 Implementation

The Isla footprint tool generates the instruction execution trace per instruction. We automate the process of generating traces from Isla for a given list of instructions. We get the stream of instructions from the official instruction set listings in the RISC-V specification manual [7] in the AsciiDoc format (*e.g.,* Listing 4).

```
0000000| rs2| rs1| 000| rd| 0110011| ADD
```

Listing 4: Example of instruction listing from the RISC-V Specification.

We construct instructions from the listings which provides concrete values for the 7-bit opcodes (*e.g.,* 0110011) and the associated operations (funct3 = 000 and funct7 = 0000000). For the source and destination registers (rs1, rs2, and rd), we use two concrete values from the GPRs in RISC-V: `zero` (x0) and any other GPR (*e.g.,* x10). We do this instead of using symbolic values here to limit the amount of path exploration in Isla. However, capturing the behaviour of the instruction with the read-only `zero` register in RISC-V ensures that the traces we capture covers this special case. For the immediate operands, we use symbolic values.

We add a new option in Isla footprint to take as input the AsciiDoc file with the instruction listings and produce the traces for all the instructions. It does so in an annotated SMT-LIB2 format as Listing 2 shows. The ISA Inspector parses the instruction traces (written in Python) to extract the ISA insights from Isla's output.

The Analyzer, implemented in Python, consumes the tables mentioned previously, and then produce the list of sensitive ISA-states.

4.1 Architecture Independent Implementation

We apply Sailor primarily on the RISC-V Sail model [17]. However, Sailor can also be used with the Sail models of other architectures. Specifically, Isla can produce instruction execution traces for Arm as well. The changes made to Isla to automate trace generation for many instructions are architecture specific. Those would need to be re-implemented for Arm. However, the parsing of the trace and the Analyzer implementation is completely architecture independent and can be used as is to analyze results for Arm. The automatic test generation uses CSR read and write template programs which are also specific to RISC-V, it would be easy to implement the register read/write template for Arm though to use the automatic test generation.

5 Evaluation

The evaluation of Sailor answers the following questions:

1. Computation time: How much time does it take to generate the ISA-level insights?
2. Accuracy of the ISA Inspector: Are there false negatives or false positives in the ISA-state detected as security-sensitive?
3. Vulnerability assessment: Can Sailor Analyzer's results be applied to analyze existing context-switching code, and potentially identify issues in it?

Evaluation setup: We ran Isla on top of Linux v6.8.0 on a server equipped with Intel Xeon Gold 5520+, with 112 cores and 504 GB RAM.

5.1 Computation Time

The ISA Inspector generates 2.8 GB of traces from Isla and feeds them to Python scripts that parse the s-expression traces and extract the insights into the tables described in Subsection 3.1, in CSV format. The Analyzer uses those CSV files as input to execute the algorithm shown in Figure 4 and determine for each architectural state whether it is security sensitive. The analysis was performed on the base ISA *i.e.,* rv64gc, along with the following extensions: vector, user-interrupts, scalar cryptography ISA extensions, performance monitoring unit, which considered 150 instructions and 255 CSRs. It takes 20 hours for generating the traces from Isla and then parsing them and performing the subsequent analysis. Most of the time is spent by Isla in its symbolic execution.

In Isla, we do not assume any values for any of the CSRs, all of them are treated as symbolic and thus multiple traces are generated for each instruction, each trace representing one possible execution path. Given the amount of exploration

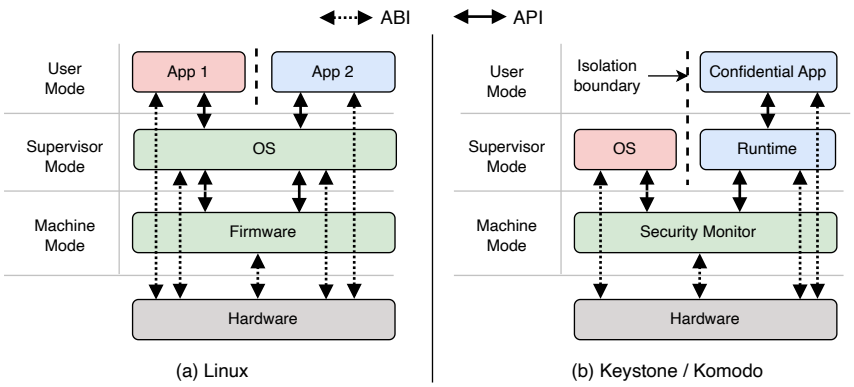

Figure 5: Overview of the analysed systems. removed h-extension frameworks, added linux.

required, the computation time is quite decent, and this analysis could be repeated even daily in order to compare against up-to-date ISA insights. Considering the deployment use-case of our ISA analysis, we believe these computation times to be reasonable.

5.2 Accuracy of the ISA Inspector

We provide no constraints to Isla regarding the initial values of the ISA-state, so Isla assumes them to be symbolic. Thus, for each instruction, Isla generates multiple traces corresponding to each execution path the instruction can take. This includes all the possible successful executions of the instruction (the ones that don't trap) as well as the unsuccessful executions of the instruction (the ones that raise a trap). To compute the instruction footprint, as specified in Subsection 3.1, we take a union of all the ISA-state read or written in each of the execution instance trace generated by Isla. Thus, we trust the Sail model and Isla to cover all cases faithfully.

The Sail model implementation generates certain spurious reads in the Isla traces especially concerning CSRs with multiple fields *e.g.,* `mcause`. This is because, to write one field of the CSR, the CSR will first be read to obtain the existing values of the remaining fields, and then will be written to reflect the old values in the unchanged fields and the new value in the changed fields. This is required when only updating a subset of the CSR's fields, but generates false reads of the CSR when the entire CSR is updated, but one field at a time. For *e.g.,* the `mcause` CSR has two fields: `IsInterrupt` which specifies whether the trap is taken due to an interrupt or not, and `cause` which specifies the type of the trap. In the trap handler implementation in the RISC-V Sail model, both fields are updated, but each field is written one at a time, generating spurious reads to `mcause` in the Isla trace. As a result, `mcause` is falsely marked as security-sensitive by Sailor for U-mode to U-mode switches as Sailor considers instructions to be dependent on it due to the spurious reads, even though it is neither accessible in U-mode, not impacts execution. This can be rectified by replacing the per field updates to complete CSR updates in the Sail model implementation wherever applicable.

The `mtvec` CSR, which contains the address of the trap handler, is also flagged as security-sensitive for a U-mode to U-mode switch. This is because all trapping instruction execution instances have a dependence on `mtvec`, so it satisfies the source and target privilege mode dependence. However, for a U-mode to U-mode switch, S-mode software and M-mode software are assumed to be trusted and not change across the context switch. So even though `mtvec` contains the address for the trap handler and is thus a critical ISA-state that must be configured properly to ensure correct behaviour, it may not necessarily require to be swapped during the context switch. Thus, the developers can consider the security-sensitive ISA-state flagged by Sailor against their threat model and take the appropriate measures in their context switch implementations.

5.3 Analysis of context switch code

We used the results from the Analyzer to analyze the context switch implementations in existing systems: Linux from StarFive VisionFive 2 [2], Keystone [39], and Komodo [29] see Figure 5. We found three distinct mishandled ISA-state that could lead to different types of security vulnerabilities. Table 4 summarizes these findings.

5.3.1 Context Switch Between Processes in Linux

We analyzed the context switch between two regular user processes running on top of the Linux OS shipped with the StarFive's VisionFive2 development board.

First, we experimentally confirmed that Linux swaps floating point state appropriately during the context switch using tests produced by the Sailor's test generator. As the board does not support the vector extension, we manually verified that Linux properly implements swapping of the vector state, which includes security sensitive control registers.

Mishandled ISA-state	Framework	Context-switch action	Potential security vulnerability
Zicntr-extension	Linux	Does not swap	Side-channel, Covert-channel, Timing-channel
`senvcfg` CSR	Keystone	Does not swap	Computational integrity, Denial-of-Service
F-extension	Keystone	Does not swap	Computational integrity*, Side-channel*, Covert-channel
`senvcfg` CSR	Komodo	Does not swap	Computational integrity, Denial-of-Service

Table 4: Mishandled ISA-state found using Sailor. We indicate the class of security consequences that mishandling of these state could lead to. (*) These were discovered and demonstrated by prior work using manual analysis of the ISA [18].

We focused next on verification of performance counters pertaining to RISC-V Zicntr extension, which Sailor reported as security-sensitive. Specifically, we experimentally tested the saving of these ISA-state for the Linux image provided with the board and found that the performance counters, among others `cycle`, `time`, and `instret`, were not sanitized or saved across context switches. Contents of these registers could be observed by U-mode processes. These high-precision timers are well-known to give rise to several security vulnerabilities, which have been studied extensively in literature [36, 37, 44, 49, 53, 56] including side-channels, timing-channels, and covert-channels. Though, by default, these performance counters are not supposed to be accessible to unprivileged user processes, and are only allowed access through the supervisor CSR `scounteren`, to be exposed through the perf tool [1] which safely isolates the measurements reported and prevents any side-channels. However, we observe that, on the VisionFive2 board, Linux allows access to these fine-grained performance counters by default to user processes.

5.3.2 Keystone Enclave Context Switching

Keystone [39] is a security monitor that enables creating trusted execution environments (TEEs) on the RISC-V platform. Keystone enclaves comprise of a trusted runtime that executes in S-mode and an enclave application that executes in U-mode. An untrusted OS executes in S-mode as usual.

We analyzed the security domain switch between the enclave and the untrusted OS as a switch from S-mode to S-mode via the security monitor in M-mode. So we query Sailor to produce security sensitive ISA state with both source and target privilege modes as S-mode. We then use those insights to experimentally verify that the context switch code in Keystone correctly saves all security sensitive state for the source security domain and similarly restores all security sensitive state for the target security domain.

Our analysis found several security-sensitive ISA-state that Keystone does not swap during the context switch.

We found that Keystone **does not swap the `senvcfg` CSR**. The `senvcfg` CSR has a `FIOM` (Fence of I/O implies fence of Memory) bitfield. The `FIOM` bit exists to handle emulated I/O for a VM running in U-mode and a hypervisor running in S-mode when paravirtualization is not available. When such a VM has multiple virtual harts, concurrent accesses to the device by the harts translate to concurrent accesses to memory due to the device emulation. However, since the VM is not aware of the device emulation, it might simply execute I/O fences (`fence io, io`) to order accesses to the device. When the `FIOM` bit is set, these I/O fence instructions lead to memory accesses also being ordered via memory fences (`fence rw, rw`).

If the FIOM bit is used by the trusted runtime in Keystone to emulate a device for the enclave application, any transitions to the untrusted OS can modify and reset this bit. If the trusted runtime transitions to the OS for unrelated reasons, it is unlikely for the runtime to enforce checks on the architectural state that it expects the security monitor to take care of saving and restoring during the context switch.

Listing 5 illustrates a scenario where a wrong FIOM configuration disrupts the correct functioning of the system, leading to potential break of computational integrity or denial-of-service. Let's assume that an untrusted hypervisor adversarially resets the FIOM bit. A dummy I/O device within the enclave that writes commands to a register mapped at address 0x1000 and then sends an inter processor interrupt (IPI) to another hart after having executed a fence IO. The command 0x43 written by hart 0 is then expected to be correctly read by hart 1. Now, in the case where the device is virtualized, this memory region could become a non-IO region. In that scenario, after the IPI is received, hart 1 could observe an incorrect stale value because the address 0x1000 is mapped into main memory and is not I/O space, and a memory fence (fence rw, rw) would be required by the memory model to ensure expected behaviour.

```
Code executed by Hart 0: // I/O emulation
// 0x1000 = memory-mapped address
// instead of I/O space
li a1, 0x1000
li a0, 0x43
// store 0x43 at address 0x1000
sw a0, 0(a1)
fence io, io    // order I/O operations
// Send IPI to Core 1

Code executed by Hart 1:
// Wait for IPI
```

```
li a1, 0x1000
ld a4, 0(a1)     // read 0x43
```

Listing 5: Example use of FIOM bit in a dummy emulated device.

Further, the OS can maliciously slow down an enclave that doesn't use the `FIOM` feature, by enabling it and requiring any I/O fences that execute inside the enclave application to be paired with memory fences, leading to denial-of-service. Neither our emulation platform, nor the StarFive VisionFive 2 board leverage this aspect of the RISC-V specification, however Keystone should properly swap the state for future platforms that might want to leverage this feature.

Further, the Floating Point extension (F-extension) in RISC-V is included in a general-purpose core (rv64gc) by default. However, Keystone does not swap the ISA state corresponding to the F-extension, introducing leaks across security domains. This includes the `fcsr` control and status register (superset of the `fflags` and `frm` CSRs) and general-purpose floating point registers *i.e.,* `f0-f31`. This opens up the attack surface due to the following potential security vulnerabilities.

The `fcsr` register has a floating point rounding mode field (`frm`). **Computational integrity** attacks can be mounted by the untrusted OS by maliciously modifying the `frm` field (floating point rounding mode) in the `fcsr`, that leads to varying computation results inside the enclave. Similarly, the general-purpose floating point registers *i.e.,* `f0-f31` can also be leveraged to breach the computational integrity of the enclave.

Side-channel and **covert-channel** attacks can also exploit the F-extension CSRs in RISC-V. These attacks can leverage the `fcsr` register, specifically the `fflags` field, which records floating-point exceptions like divide-by-zero. Instead of raising traps, the processor updates `fflags`, requiring software to proactively check for exceptions. These attacks can also leverage the `frm` field and the `f0-f31` registers.

Prior work has already discovered two of the vulnerabilities we describe above (the side-channel and computational integrity attacks through the `fcsr` register [18]) and reported them to Keystone [5]. Although Keystone is currently under active development, the authors have neither responded to the report nor introduced a fix in the framework. [3]

5.3.3 Komodo Enclave Context Switching

Komodo is a verified security monitor to create enclaves [29]. Komodo (initially developed on Arm) has been ported to RISC-V and verified by a framework called Serval [45]. It has a model similar to Keystone, with a security domain switch occurring from S-mode to S-mode.

[3] We reproduce the side-channel and computational integrity attacks by prior work [18] and also provide a proof-of-concept implementation for new side-channel and covert-channel attacks that leverage the `f0-f31` registers.

The Serval framework verifies Komodo for the RISC-V integer, multiplication, and Zicsr extensions. Considering only these extensions, we still found that Komodo does not swap the **`senvcfg` CSR**.

However, we note that the `senvcfg` CSR was introduced into the RISC-V privileged specification v1.12, which was ratified after the publication of the Serval paper [7, 45]. As such, Komodo was perfectly correct at publication-time, however later change in the specification rendered the context-switch code technically incomplete and potentially unsecure. This brings an interesting case of maintainability, and keeping up with changing ISA specification. This case shows that it is important to automate detection and testing of security-sensitive ISA-state as it is a process that must be kept up-to-date with the ISA changes.

5.3.4 ISA extensions

While Keystone doesn't claim support for platforms with extensions such as vector (V), user interrupt and exception handling (N), it takes no preventative measures to detect an unsupported platform and halt further execution or disable those extensions. Similarly, Komodo also does not take any measures to prevent being incorrectly deployed on unsupported platforms. The task of deploying a system only on suitable platforms is left for the system operators/platform providers to take care of. Adding such checks can improve defense-in-depth and proactively prevent exploitation of any security vulnerabilities that stem from architectural extensions not supported by security-critical software like security monitors, especially for platforms like RISC-V that encourage custom extensions.

Further, it can be beneficial w.r.t. security to adhere to the following practices while designing new ISA extensions. ISA-state previously not classified as security-sensitive must not be affected by the extension to be reclassified as security-sensitive. If that is the case, then Sailor needs to be reapplied on the entire ISA specification to correctly determine security-sensitive ISA-state. Newly introduced ISA-state should not be security-sensitive when the extension is disabled. This is important to ensure that privileged software that does not support certain extensions can still correctly execute on the platform by disabling those extensions.

6 Related Work

Prior work includes manual analysis of the ISA specification to find exploitable bugs in context switch implementations [18, 26, 38, 40, 52]. Manual analysis of the entire ISA specification can trivially become erroneous, similar to introducing bugs in context switch implementations in the first place. These prior works typically focus on particular systems to analyze or finding bugs related to particular ISA extensions (*e.g.,* Floating Point Unit [18]) to limit the scope of the

required manual ISA-analysis.

Other related work targets general correctness of emulators/virtualization [33,43,50,54]. These works target checking whether single instruction semantics being emulated are faithfully emulating real hardware. While we also analyze ISA properties, and use symbolic execution, we target a different problem: finding out the ISA-state that must be saved and cleared across context switches. There are several similarities and differences between these works and Sailor in regards to the focus on the ISA semantics. For instance, Examiner [33] also focuses towards finding the various control flows of the instruction, similarly to how Isla generates one trace per execution path of an instruction. However, Sailor leverages Isla's trace generation to more specifically extract a union of all the ISA-state read or written during any of the execution, rather than focusing on the control flows itself. Martignoni et. al. [43] use a `high-fidelity` emulator that is expected to have less bugs for verifying general purpose emulators as a proxy for using hardware specifications, due to lack of availability of hardware specifications. We leverage the availability of the Sail specification to not suffer through this limitation. Cardinal pill testing again focuses on verifying emulators against the hardware specifications [50]. Sail has similarly gained further momentum with recent work using Sail specifications to verify hypervisors [27], albiet in an orthogonal research direction than Sailor.

Some privileged software are formally verified for functionality and security properties, such as non-interference, often requiring multiple years of effort for extensive verification [29,30,34]. However, even that is not always enough to guarantee strict isolation at the ABI layer as these works write their own system specifications which are usually a subset of the actual ISA. This is done in order to make the verification feasible, since the work tends to be strenuous and if the state explored is not limited, then the verifiers or provers can quickly face path explosions. For instance, prior work verifies security monitor software for RISC-V and x86-32 architectures, but model only the integer (I), multiplication (M) and control and status register (Zicsr) instruction sets for RISC-V, and only GPRs for x86-32 [45].

Sail has an Ocaml parser that can be used with any of the different backends also written in Ocaml. The backends are for compiling Sail into the C/Ocaml emulators, or producing theorem prover definitions for Coq, Hol4, Isabelle, etc. In the existing implementation, the Sail parser cannot directly produce the type of ISA insights we extract with the Scanner. There are existing efforts to create a new backend that works with the Ocaml parser to extract ISA-related information from Sail [46]. The type of ISA-related information extracted includes name and operands of instructions, their description and register names, descriptions and sizes. The backend is meant to be simple and serve as a stepping stone for researchers to be able to extend that into more sophisticated use-cases. While Sailor is focused towards extracting more specific information from Sail towards the pre-defined goal of secure context switch implementations, compared to the work described above, both the works are aligned in making use of Sail to automatically retrieve ISA-related information and replace manual efforts of browsing through ISA specifications.

Prior work involves attacks at the API-layer (as Figure 1 shows) which includes exploiting missing pointer checks, buffer overflows, etc. that can cause memory leaks or breach integrity of security domains [28,52]. Our work specifically focuses on the ABI-layer and not the API-layer, and thus requires mitigation strategies to prevent attacks at the API-layer to work in tandem. Similarly, our work does not target preventing micro-architectural attacks (cache side-channels, speculative execution attacks [35,42,55]) and requires separately applying mitigation strategies for those as well.

7 Future Scope

While RISC-V serves as a suitable candidate for demonstration, the challenge of manually analyzing complex ISAs like Arm and x86 is significantly greater. Sailor can significantly expedite the process of identifying security-sensitive ISA-state for those ISAs. Extending Sailor to support other ISAs, especially Arm, since Isla can generate traces for Arm, remains a future work.

The Analyzer's output *i.e.,* security-sensitive ISA-state, can be leveraged by verification frameworks such as Serval [45] to ensure that they prove security properties on the context switch code with an ISA model including all the security-sensitive ISA-state. The Analyzer's output can also be used to exclude the ISA-state that is not security-sensitive from the verification frameworks's ISA specification model.

Sailor can be integrated with the CI/CD pipelines of security monitors or other privileged software to enforce checks on the context switch implementation against Sailor's results. This can be even more useful for software under active development as Sailor can proactively capture insights from the latest ISA specifications or as the software starts supporting more ISA extensions.

8 Conclusion

Our work concentrated on an often overlooked area of the attack surface: the ABI-layer. The increasing complexity of ISA specifications make it more difficult to write secure context switch code. We introduced a tool, Sailor, that leverages machine-readable ISA specifications written in the Sail language and Isla, symbolic execution engine tool for Sail code, to automatically identify which ISA-state must be swapped during a context switch across different security domains. Consequently, our tool replaces the strenuous task of manually navigating through hundreds or thousands of pages of ISA

specifications to figure out which ISA-state to swap during the context switch. We detected multiple mishandled ISA-state across different systems for the RISC-V ISA using the results from our tool.

As ISAs continue to evolve, it is crucial to automate the task of analysing the latest specifications. We believe that as ISAs continue to evolve and increase in complexity, tools like Sailor are crucial for automating tedious manual tasks. Further, it is important to keep the machine-readable ISA specifications in Sail up to date as well, rather than just relying on prose-style documentations. We encourage hardware vendors to provide Sail models for their machines, especially when the hardware implements custom ISA-extensions, to promote comprehensive security analysis of the hardware.

9 Open Science

In line with the USENIX Security open science policy, we have open-sourced the project. This will facilitate reproducibility and enable the research community to build upon our findings. By making the source code publicly available, we aim to foster collaboration and encourage further innovation. The Open Science approach ensures the transparency and accessibility of our research.

10 Ethics Considerations

This research resulted in the identification and responsible disclosure of vulnerabilities in existing academic research systems as well as software dispatched by popularly used development boards. The developers of the affected systems are stakeholders. Our work provided these systems with information to enhance the security and robustness of their research platforms. We have already disclosed the vulnerabilities found as a result of this research to through appropriate channels. The research community benefits from improved system security and awareness of potential vulnerabilities. As these systems primarily serve research and development purposes, there is no direct harmful impact on the developers of the affected system or the users.

Acknowledgments

We thank Silvio Dragone (IBM Research Zurich), Edouard Bugnion (EPFL, Switzerland), Michael Le, Elaine Palmer (IBM T. J. Watson Research Center), Lennard Gäher (MPI-SWS Germany), as well as Charly Castes (EPFL, Switzerland) and Adrien Ghosn (Azure Research, Cambridge) for their valuable comments and feedback. This work was conducted as a part of IBM Research's project, Assured Confidential Computing (ACE) for RISC-V.

References

[1] Linux perf tool. https://perfwiki.github.io/main/.

[2] Starfive visionfive 2. https://github.com/starfive-tech/VisionFive2.

[3] Starfive visionfive2 linux security vulnerability github issue. https://github.com/starfive-tech/VisionFive2/issues/114, year=2025,.

[4] Arm® architecture reference manual, armv7-a and armv7-r edition. https://developer.arm.com/documentation/ddi0406/latest/, 2018.

[5] Missing sanitisation of fpu rounding mode and exception flags on enclave entry/exit. https://github.com/keystone-enclave/keystone-sdk/issues/72, 2021.

[6] Intel trust domain extensions (TDX) security review, google cloud security and google project zero, 2023.

[7] RISC-V technical specifications. https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications, 2024.

[8] Compiled sail isa into the intermediate representation for the isla symbolic execution tool. https://github.com/rems-project/isla-snapshots, 2025.

[9] Intel software guard extensions – get started with the SDK. https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/get-started.html, 2025.

[10] Intel trust domain extensions (TDX). https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html, 2025.

[11] Keystone security vulnerability github issue. https://github.com/keystone-enclave/keystone/issues/483, 2025.

[12] Microsoft open enclave SDK. https://openenclave.io/sdk/, 2025.

[13] RISC-V ratified extensions. https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154732/Ratified+Extensions, 2025.

[14] RISC-V instruction set manual. https://github.com/riscv/riscv-isa-manual, 2025.

[15] The sail isa specification language. https://github.com/rems-project/sail, 2025.

[16] Sail model for the Arm ISA. https://github.com/rems-project/sail-arm, 2025.

[17] Sail model for the RISC-V ISA. https://github.com/riscv/sail-riscv, 2025.

[18] Fritz Alder, Jo Van Bulck, Jesse Spielman, David Oswald, and Frank Piessens. Faulty point unit: ABI poisoning attacks on trusted execution environments. *Digital Threats*, 3(2), February 2022.

[19] ARM. Arm memory protection unit. https://developer.arm.com/documentation/100166/0001/Memory-Protection-Unit/About-the-MPU.

[20] ARM. Building a secure system using trustzone technology. *White Paper, April*, 2009.

[21] ARM. Arm® architecture reference manual armv8, for armv8-a architecture profile. https://developer.arm.com/documentation/ddi0487/aa/?lang=en, 2017.

[22] ARM. Arm® architecture reference manual for a-profile architecture. https://developer.arm.com/documentation/ddi0487/latest/, 2024.

[23] Alasdair Armstrong, Brian Campbell, Ben Simner, Christopher Pulte, and Peter Sewell. Isla: Integrating full-scale isa semantics and axiomatic concurrency models. In *Computer Aided Verification: 33rd International Conference, CAV 2021, Virtual Event, July 20–23, 2021, Proceedings, Part I*, page 303–316, Berlin, Heidelberg, 2021. Springer-Verlag.

[24] Clark W. Barrett, Aaron Stump, and Cesare Tinelli. The smt-lib standard version 2.0. 2010.

[25] Andrew Baumann. Hardware is the new software. HotOS '17, page 132–137, New York, NY, USA, 2017. Association for Computing Machinery.

[26] Andrea Biondo, Mauro Conti, Lucas Davi, Tommaso Frassetto, and Ahmad-Reza Sadeghi. The guard's dilemma: efficient code-reuse attacks against intel sgx. In *Proceedings of the 27th USENIX Conference on Security Symposium*, SEC'18, page 1213–1227, USA, 2018. USENIX Association.

[27] Charly Castes, François Costa, Nate Foster, Thomas Bourgeat, and Edouard Bugnion. Lightweight hypervisor verification: Putting the hardware burger on a diet. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*, HotOS '25, page 27–33, New York, NY, USA, 2025. Association for Computing Machinery.

[28] Stephen Checkoway and Hovav Shacham. Iago attacks: why the system call api is a bad untrusted rpc interface. ASPLOS '13, page 253–264, New York, NY, USA, 2013. Association for Computing Machinery.

[29] Andrew Ferraiuolo, Andrew Baumann, Chris Hawblitzel, and Bryan Parno. Komodo: Using verification to disentangle secure-enclave hardware from software. In *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP '17, page 287–305, New York, NY, USA, 2017. Association for Computing Machinery.

[30] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. CertiKOS: An extensible architecture for building certified concurrent os kernels. In *OSDI*, pages 653–669. USENIX Association, 2016.

[31] Jack Tigar Humphries, Kostis Kaffes, David Mazières, and Christos Kozyrakis. A case against (most) context switches. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, HotOS '21, page 17–25, New York, NY, USA, 2021. Association for Computing Machinery.

[32] Intel. Intel software guard extensions (intel sgx). https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html, 2023.

[33] Muhui Jiang, Tianyi Xu, Yajin Zhou, Yufeng Hu, Ming Zhong, Lei Wu, Xiapu Luo, and Kui Ren. Examiner: automatically locating inconsistent instructions between real devices and cpu emulators for arm. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, page 846–858, New York, NY, USA, 2022. Association for Computing Machinery.

[34] Gerwin Klein, June Andronick, Kevin Elphinstone, Gernot Heiser, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. sel4: formal verification of an operating-system kernel. *Commun. ACM*, 53(6):107–115, June 2010.

[35] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: exploiting speculative execution. *Commun. ACM*, 63(7):93–101, June 2020.

[36] Paul C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In *Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.

[37] Bhargav Achary Dandpati Kumar, Sai Chandra Teja R, Sparsh Mittal, Biswabandan Panda, and C. Krishna Mohan. Inferring dnn layer-types through a hardware performance counters based side channel attack. In *Proceedings of the First International Conference on AI-ML Systems*, AIMLSystems '21, New York, NY, USA, 2021. Association for Computing Machinery.

[38] Stella Lau, Thomas Bourgeat, Clément Pit-Claudel, and Adam Chlipala. Specification and verification of strong timing isolation of hardware enclaves. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 1121–1135, New York, NY, USA, 2024. Association for Computing Machinery.

[39] Dayeol Lee, David Kohlbrenner, Shweta Shinde, Krste Asanović, and Dawn Song. Keystone: an open framework for architecting trusted execution environments. In *Proceedings of the Fifteenth European Conference on Computer Systems*, EuroSys '20, New York, NY, USA, 2020. Association for Computing Machinery.

[40] Jaehyuk Lee, Jinsoo Jang, Yeongjin Jang, Nohyun Kwak, Yeseul Choi, Changho Choi, Taesoo Kim, Marcus Peinado, and Brent Byunghoon Kang. Hacking in darkness: return-oriented programming against secure enclaves. In *Proceedings of the 26th USENIX Conference on Security Symposium*, SEC'17, page 523–539, USA, 2017. USENIX Association.

[41] Chuanpeng Li, Chen Ding, and Kai Shen. Quantifying the cost of context switch. In *Proceedings of the 2007 Workshop on Experimental Computer Science*, ExpCS '07, page 2–es, New York, NY, USA, 2007. Association for Computing Machinery.

[42] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown: reading kernel memory from user space. In *Proceedings of the 27th USENIX Conference on Security Symposium*, SEC'18, page 973–990, USA, 2018. USENIX Association.

[43] Lorenzo Martignoni, Stephen McCamant, Pongsin Poosankam, Dawn Song, and Petros Maniatis. Path-exploration lifting: hi-fi tests for lo-fi emulators. *SIGPLAN Not.*, 47(4):337–348, March 2012.

[44] Hoda Naghibijouybari, Ajaya Neupane, Zhiyun Qian, and Nael Abu-Ghazaleh. Rendered insecure: Gpu side channel attacks are practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, page 2139–2153, New York, NY, USA, 2018. Association for Computing Machinery.

[45] Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. Scaling symbolic evaluation for automated verification of systems code with serval. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 225–242, New York, NY, USA, 2019. Association for Computing Machinery.

[46] Ventana Micro Systems Paul Clarke. RISC-V instruction information parsing and storage for Sail. RISC-V Summit, 2023.

[47] Christian Priebe, Divya Muthukumaran, Joshua Lind, Huanzhou Zhu, Shujie Cui, Vasily A. Sartakov, and Peter Pietzuch. Sgx-lkl: Securing the host os interface for trusted execution, 2020.

[48] Michael Sammler, Angus Hammond, Rodolphe Lepigre, Brian Campbell, Jean Pichon-Pharabod, Derek Dreyer, Deepak Garg, and Peter Sewell. Islaris: verification of machine code against authoritative isa semantics. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2022, page 825–840, New York, NY, USA, 2022. Association for Computing Machinery.

[49] Ron Segev and Avi Mendelson. The use of performance-countersto perform side-channel attacks. In *Cyber Security, Cryptology, and Machine Learning: 7th International Symposium, CSCML 2023, Be'er Sheva, Israel, June 29–30, 2023, Proceedings*, page 216–233, Berlin, Heidelberg, 2023. Springer-Verlag.

[50] Hao Shi, Abdulla Alwabel, and Jelena Mirkovic. Cardinal pill testing of system virtual machines. In Kevin Fu and Jaeyeon Jung, editors, *Proceedings of the 23rd USENIX Security Symposium, San Diego, CA, USA, August 20-22, 2014*, pages 271–285. USENIX Association, 2014.

[51] Richard Uhlig, Gil Neiger, Dion Rodgers, Amy L. Santoni, Fernando C. M. Martins, Andrew V. Anderson, Steven M. Bennett, Alain Kägi, Felix H. Leung, and Larry Smith. Intel Virtualization Technology. *Computer*, 38(5):48–56, 2005.

[52] Jo Van Bulck, David Oswald, Eduard Marin, Abdulla Aldoseri, Flavio D. Garcia, and Frank Piessens. A tale of two worlds: Assessing the vulnerability of enclave shielding runtimes. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, page 1741–1758, New York, NY, USA, 2019. Association for Computing Machinery.

[53] Junyi Wei, Yicheng Zhang, Zhe Zhou, Zhou Li, and Mohammad Abdullah Al Faruque. Leaky dnn: Stealing deep-learning model secret with gpu context-switching

side-channel. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 125–137, 2020.

[54] Qiuchen Yan and Stephen McCamant. Fast pokeemu: Scaling generated instruction tests using aggregation and state chaining. In *Proceedings of the 14th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, VEE '18, page 71–83, New York, NY, USA, 2018. Association for Computing Machinery.

[55] Yuval Yarom and Katrina Falkner. Flush+reload: a high resolution, low noise, l3 cache side-channel attack. In *Proceedings of the 23rd USENIX Conference on Security Symposium*, SEC'14, page 719–732, USA, 2014. USENIX Association.

[56] Yicheng Zhang, Carter Slocum, Jiasi Chen, and Nael Abu-Ghazaleh. It's all in your head(set): Side-channel attacks on AR/VR systems. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3979–3996, Anaheim, CA, August 2023. USENIX Association.